COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure and Algorithms	**SECTION:**	CPE 21S3
DATE PERFORMED:	9/15/26	**DATE SUBMITTED:**	9/15/26
NAME:	Vince Andrew Urbano	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 8.1 **Sorting Algorithm Part 2**

1. PROCEDURE OUTPUT

Code + Console Screenshot

```
const int maxSize = 100;

void displayFunc(int arr[]);

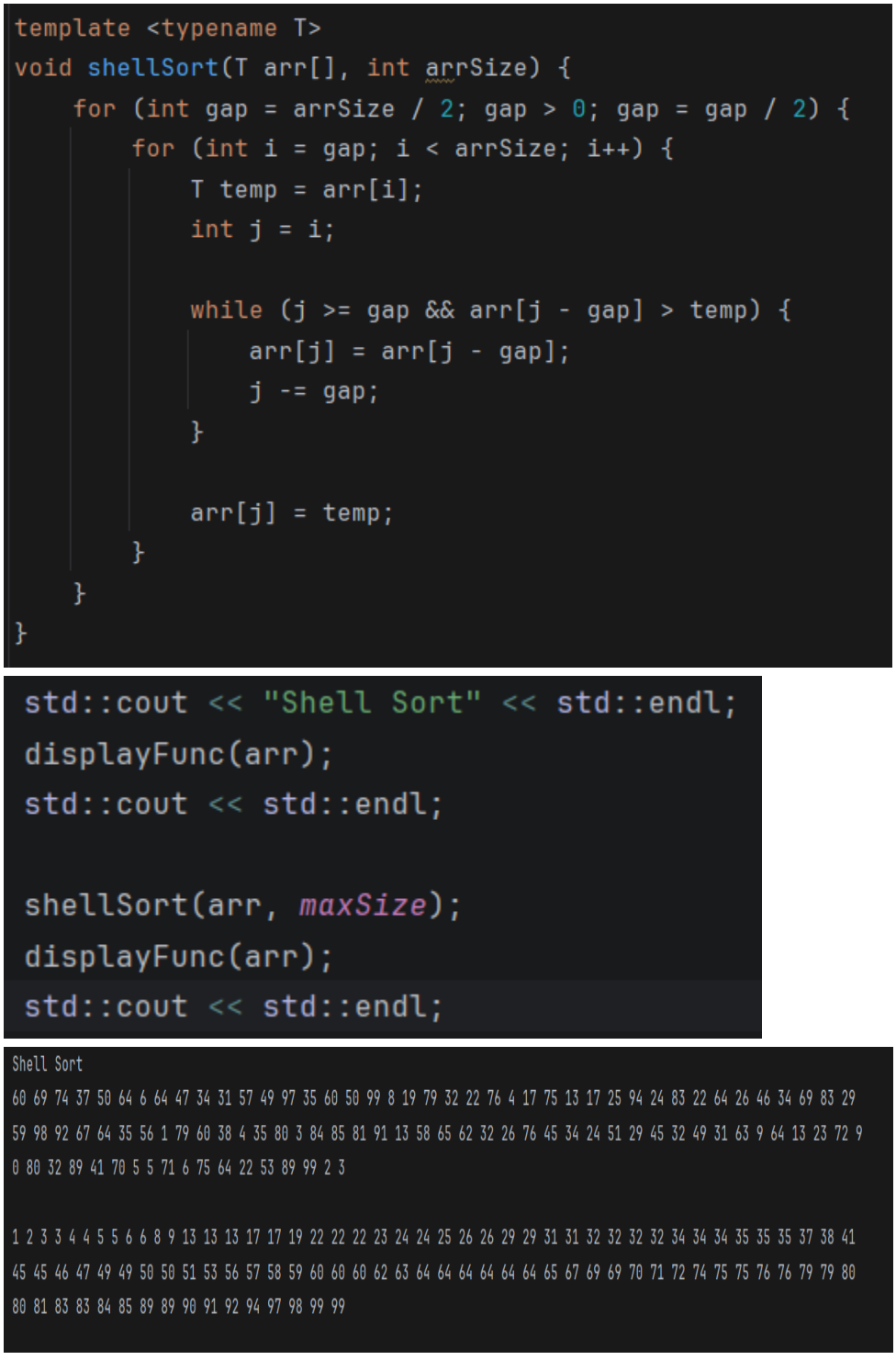
int main() {
    int arr[maxSize];
    int arr2[maxSize];
    int arr3[maxSize];
    srand( Seed: time(0));

    for (int i = 0; i < maxSize; i++) {
        arr[i] = rand() % 100;
        arr2[i] = arr[i];
        arr3[i] = arr[i];
    }
```

```
60 69 74 37 50 64 6 64 47 34 31 57 49 97 35 60 50 99 8 19 79 32 22 76 4 17 75 13 17 25 94 24 83 22 64 26 46 34 69 83 29
59 98 92 67 64 35 56 1 79 60 38 4 35 80 3 84 85 81 91 13 58 65 62 32 26 76 45 34 24 51 29 45 32 49 31 63 9 64 13 23 72 9
0 80 32 89 41 70 5 5 71 6 75 64 22 53 89 99 2 3
```

Observations	First we implement the size of the array and use the rand() function to generate 100 random integer values.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	
Observations	The shell sort is checking each value within its gap. It also divides by two each time it has

	completed the loop which makes the gap shorter and checks every single value one by one.

Table 8-2. Shell Sort Technique

Code + Console Screenshot	

```
template <typename T>
void merge(T arr[], int left, int mid, int right) {
    int num1 = mid - left + 1;
    int num2 = right - mid;

    T* Arr = new T[num1];
    T* Arr2 = new T[num2];

    for (int i = 0; i < num1; i++) {
        Arr[i] = arr[left + i];
    }
    for (int j = 0; j < num2; j++) {
        Arr2[j] = arr[mid + 1 + j];
    }

    int i = 0;
    int j = 0;
    int k = left;

    while (i < num1 && j < num2) {
        if (Arr[i] <= Arr2[j]) {
            arr[k] = Arr[i];
            i++;
        } else {
            arr[k] = Arr2[j];
            j++;
        }
        k++;
    }
```

```
    while (i < num1) {
        arr[k] = Arr[i];
        k++;
        i++;
    }

    while (j < num2) {
        arr[k] = Arr2[j];
        k++;
        j++;
    }

    delete[] Arr;
    delete[] Arr2;
}

template <typename T>
void mergeSort(T arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, right: mid);
        mergeSort(arr, left: mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

```
std::cout << "Merge Sort" << std::endl;
displayFunc(arr2);
std::cout << std::endl;
mergeSort(arr2, left: 0 , right: maxSize -1);
displayFunc(arr2);
std::cout << std::endl;
```

	Merge Sort 60 69 74 37 50 64 6 64 47 34 31 57 49 97 35 60 50 99 8 19 79 32 22 76 4 17 75 13 17 25 94 24 83 22 64 26 46 34 69 83 29 59 98 92 67 64 35 56 1 79 60 38 4 35 80 3 84 85 81 91 13 58 65 62 32 26 76 45 34 24 51 29 45 32 49 31 63 9 64 13 23 72 9 0 80 32 89 41 70 5 5 71 6 75 64 22 53 89 99 2 3 1 2 3 3 4 4 5 5 6 6 8 9 13 13 13 17 17 19 22 22 22 23 24 24 25 26 26 29 29 31 31 32 32 32 32 34 34 34 35 35 35 37 38 41 45 45 46 47 49 49 50 50 51 53 56 57 58 59 60 60 60 62 63 64 64 64 64 64 64 65 67 69 69 70 71 72 74 75 75 76 76 79 79 80 80 81 83 83 84 85 89 89 90 91 92 94 97 98 99 99
Observations	The program divides the array until it can no longer be divided. After dividing, it is the merged and compared and this is where it will start swapping places.,

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	

```
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            std::swap( [&] arr[i], [&] arr[j]);
        }
    }
    std::swap( [&] arr[i + 1], [&] arr[high]);
    return i+1;
}

template <typename T>
void quickSort(T arr[], int low, int high) {
    if (low < high) {
        int pivot = partition(arr, low, high);
        quickSort(arr, low, high: pivot -1);
        quickSort(arr, low: pivot + 1, high);
    }
}
```

```
    std::cout << "Quick Sort" << std::endl;
    displayFunc(arr3);
    std::cout << std::endl;
    quickSort(arr3, low: 0 , high: maxSize -1);
    displayFunc(arr3);
    std::cout << std::endl;
}
```

	Quick Sort 60 69 74 37 50 64 6 64 47 34 31 57 49 97 35 60 50 99 8 19 79 32 22 76 4 17 75 13 17 25 94 24 83 22 64 26 46 34 69 83 29 59 98 92 67 64 35 56 1 79 60 38 4 35 80 3 84 85 81 91 13 58 65 62 32 26 76 45 34 24 51 29 45 32 49 31 63 9 64 13 23 72 9 0 80 32 89 41 70 5 5 71 6 75 64 22 53 89 99 2 3 1 2 3 3 4 4 5 5 6 6 8 9 13 13 13 17 17 19 22 22 22 23 24 24 25 26 26 29 29 31 31 32 32 32 32 34 34 34 35 35 35 37 38 41 45 45 46 47 49 49 50 50 51 53 56 57 58 59 60 60 60 62 63 64 64 64 64 64 64 65 67 69 69 70 71 72 74 75 75 76 76 79 79 80 80 81 83 83 84 85 89 89 90 91 92 94 97 98 99 99
Observations	The program checks a value from the array and it swaps places. If the value is smaller than the pivot, it will move to the left and if it is larger it will move to the right.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.
Yes, in this code I have combined the quick sort and the shell sort.
Code:

```
8   template <typename T>
9   int partition(T arr[], int low, int high) {
10      T pivot = arr[high];
11      int i = low - 1;
12
13      for (int j = low; j < high; j++) {
14          if (arr[j] <= pivot) {
15              i++;
16              std::swap( [&] arr[i], [&] arr[j]);
17          }
18      }
19      std::swap( [&] arr[i + 1], [&] arr[high]);
20      return i+1;
21  }
```

```cpp

template <typename T>
void shellSort(T arr[], int low, int high) {
    int subSize = high - low + 1;
    for (int gap = subSize / 2; gap > 0; gap /= 2) {
        for (int i = low + gap; i <= high; i++) {
            T temp = arr[i];
            int j = i;
            while (j >= low + gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }
            arr[j] = temp;
        }
    }
}

```

```cpp
template <typename T>
void supplementarySort(T arr[], int left, int right) {
    if (left < right) {
        int pivot = partition(arr, low: left, high: right);

        shellSort(arr, low: left, high: pivot - 1);
        shellSort(arr, low: pivot + 1, high: right);
    }
}


#endif  #ifndef supplementary_h
```

Main:

```
const int maxSize = 100;

void displayFunc(int arr[]);

int main() {
    int arr[maxSize];
    srand( Seed: 0);

    for (int i = 0; i < maxSize; i++) {
        arr[i] = rand() % 100;
    }
    displayFunc(arr);
    std::cout << std::endl;
    supplementarySort(arr, left: 0, right: maxSize - 1);
    displayFunc(arr);

}

void displayFunc(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

Output:

```
38 19 38 37 55 97 65 85 50 12 53 0 42 81 37 21 45 85 97 80 76 91 55 6 57 23 81 40 25 78 46 90 40 87 7 37 11 17 56 67 33
78 23 87 97 84 12 11 78 66 29 4 79 5 88 49 29 76 31 64 14 36 28 2 52 4 37 56 98 72 97 13 83 3 60 42 47 75 71 4 73 52 19
4 39 86 4 37 23 35 33 93 20 74 83 61 24 65 69 30

0 2 3 4 4 4 4 4 5 6 7 11 11 12 12 13 14 17 19 19 20 21 23 23 23 24 25 28 29 29 30 31 33 33 35 36 37 37 37 37 37 38 38 39
40 40 42 42 45 46 47 49 50 52 52 53 55 55 56 56 57 60 61 64 65 65 66 67 69 71 72 73 74 75 76 76 78 78 78 79 80 81 81 83
83 84 85 85 86 87 87 88 90 91 93 97 97 97 97 98
```

Code Analysis:
First we start with the header file. In the header file we have the first function named as partition. The function of partition is to select the last element as the pivot for our quicksort program. It is also responsible for swapping the places of a small value element and a large value element. It will move the small value to the left side of the pivot and the largest will be swapped to the right of the pivot. Next is the shell sort function, it is responsible for checking the array based on the gap. Each time it completes a loop, the gap will become smaller until it reaches the zero. Lastly is supplementarySort function, it is the combination of the quick and shell sort which is executed on the main drive.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

What sorting algorithm will give you the fastest time performance?:
The sorting algorithm that will give the fastest time performance is insertion sort. This array has a small number of elements and insertion will be the best choice as a sorting algorithm as it checks the array one by one accurately.

Why can merge sort and quick sort
have O(N • log N) for their time complexity?
Merge and quick sort can have O(N • log N) as their time complexity because it involves dividing the array into different parts. For example in merge sort, it divides the arrays into two until it can no longer be divided.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea Generation and concept explanation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Gemini to see the idea of the task and explain how each sorting differ from the others.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.